System Call for Processes Management

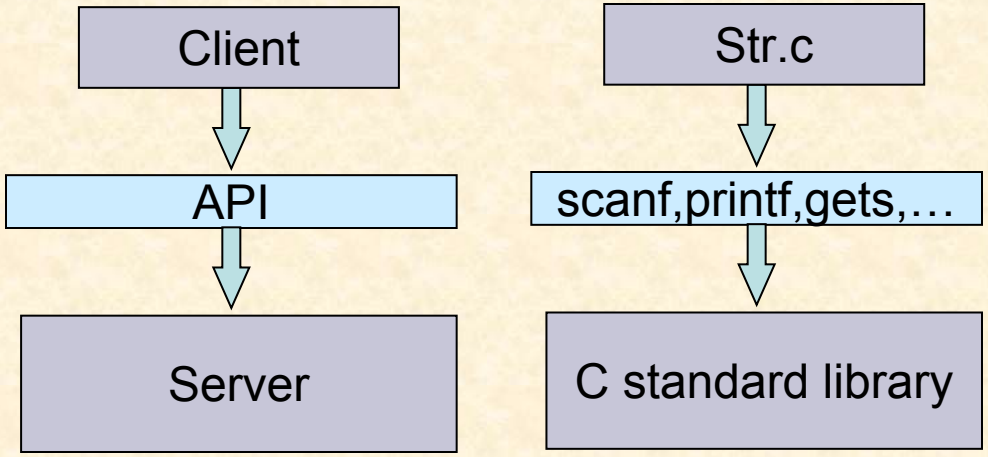
Ch5

Sk Subidh Ali
IIT Bhilai

Application Program Interface

- API: A set of rules or protocols that enables software applications to communicate with each other

These are basically a set of functions provided by the service provider




```
#include <stdio.h>
int main()
{ char str[20];
 scanf("%s",str);
 printf("%d", strlen(str));
  i=i+1;
  }
}
```

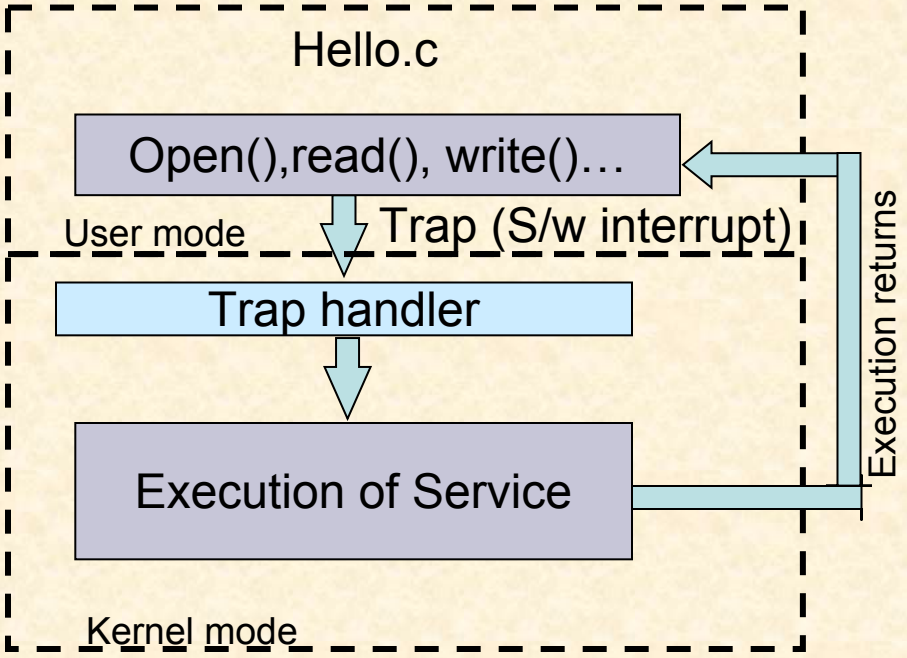
Str.c

System call

- System calls are the set of APIs provided by operating system to invoke its services Trap/software interrupt
- Invoking system call:
  - Processor to shift from user mode to privileged mode
  - Once system call completes, execution return to application process in user model

```
#include <stdio.h>
int main()
{ int fd;
  char A[]="System"
fd=open ("sk.c",o_RDWR);
write(fd,A,6);
 close(fd)
  }
}
```

Hello.c



System call for creating process

- ***int exit(int status) :***
  - Current process terminates
  - On success status value become 0
- ***int kill(int pid, int sig)***
  - Send SIGNAL in sig to process pid
  - Generally used to kill a process
  - Example
    - kill(1234, SIGTERM);
- *int wait(int *status)*
  - *Parent process blocked until child completes*

System call for creating process

- ***int for(void) :***
  - Create new process that is exact copy of current one
  - Returns process ID of new process in “parent”
  - Returns 0 in “child”
  - Booting start init() process of OS, which forks all other processes

fork() in action

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

fork() in action

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

fork() in action

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

A child process will be created and put into ready state

```
$ cc P1.c –o P1
$./P1
```

P1

fork() in action

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

rc=pid of child which is >0

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

rc=0

P1

Who will run from where?

Child of P1

If fork() is successful than $rc \not< 0$

Therefore execution will start from *else if*

fork() in action

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

rc=pid of child is >0

```
$ cc P1.c –o P1
$./P1
```

P1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

Child of P1

Consider parent is running now

fork() in action

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

rc=pid of child is >0

```
$ cc P1.c –o P1
$./P1
$Parent of 2961 (pid 5678)
```

Child of P1

Consider parent is running now

fork() in action

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

rc=0

```
$ cc P1.c –o P1
$./P1
$Parent of 2961 (pid 5678)
```

Child of P1

Consider parent is running now

fork() in action

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) getpid());
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  printf("hello (pid:%d)\n", (int) getpid());
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child (new process)
    printf("child (pid:%d)\n", (int) get
  } else {
    // parent goes down this path (main)
    printf("parent of %d (pid:%d)\n",
           rc, (int) getpid());
  }
  return 0;
}
```

rc=0

Child of P1

```
$ cc P1.c –o P1
$./P1
$Parent of 2961 (pid 5678)
$Child of (pid: 2961)
```

Consider parent is running now

System call for creating process

- ***exec():***
  - Runs executables with arguments
  - there are six variants of exec(): execl(), execlp(), execle(), execv(), execvp(), and execvpe()
  - Child process will replace the parent process. Whatever coder after exec call will not be executed
  - Example:
    - int execvp (char *prog, char **argv);
    - Search PATH for prog, use current environment
    - - int execlp (char *prog, char *arg, ...);
    - List arguments one at a time, finish with NULL

exec in action

P1.c

```
#include<stdio.h>

int main()
{
printf("I am P1");
}
```

P2.c

```
#include<stdio.h>
#include<unistd.h>

int main(){
  printf("\nExecuting P1 using exec\n");
  execl("./P1", "", NULL);
  perror("Exec failed");
  printf("\n Exec completes");
  }
```

Will never execute as child process will replace this code

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P2
```

```
$Executing P1 using exec
   I am P1
```

Parent's memory image is overwritten by the new executable in exec(); it includes code, data, stack, heap, everything.

exec() in action

P1.c

```
#include<stdio.h>

int main()
{
printf("I am P1");
}
```

P2.c

```
#include<stdio.h>
#include<unistd.h>

int main(){
  printf("\nExecuting P1 using exec\n");
  execl("./P1", "", NULL);
  perror("Exec failed");
  printf("\n Exec completes");
  }
```

Will never execute as child process will replace this code

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P2
```

```
$Executing P1 using exec
   I am P1
```

Parent's memory image is overwritten by the new executable in exec(); it includes code, data, stack, heap, everything.

Combining fork() and exec()

Linux shell is an example of combined application

```
$cc P1.c↲
$wc P1.c↲
$exit ↲
```

why doesn't shell exec command directly? Why fork a child?

```
while (1) {
    printf("$ ");
    fgets(cmd, sizeof(cmd), stdin);
    cmd[strcspn(cmd, "\n")] = '\0';
     if (strcmp(cmd, "exit") == 0)
        break;
    pid_t pid = fork();
    if (pid == 0) { // Child process
        execlp(cmd, cmd, (char *)NULL);
        perror("Command not found");
        return 1;
    } else {    // Parent process
        wait(NULL);}}
return 0;}
```

Read user commands:
cc P1.c goes as input

Check if the command is exit

Create a child process to take care of the command

Child process run the command using exec()

Parent waits until child finishes then continue in infinite loop

Terminal.
c

System call for creating process

- ***int exit(int status) :***
  - Current process terminates
  - On success status value become 0
- ***int kill(int pid, int sig)***
  - Send SIGNAL in sig to process pid
  - Generally used to kill a process
  - Example
    - kill(1234, SIGTERM);
- *int wait(int *status)*
  - *Parent process blocked until child completes*

Next Class: Programming (bring laptops)
Next theory class: Context Switch: Ch6 Remzi